The Journal of Open Source Software

A Sparse Matrix parallel solver for OpenCL

Vishakh Begari [1]

1 Independent Researcher, Germany

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Matthew Feickert

Reviewers:

- @yurlungur
- @Brookluo

Submitted: 05 June 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

PIRO is an open-source C++/OpenCL software for performing high-performance numerical simulations. It provides modular solvers, supports various sparse formats (CSR, COO), and offers kernel-level customization for advanced users. PIRO is designed for research and production workflows requiring scalable parallel matrix computations.

Statement of Need

Compressed Sparse Row (CSR) is a space efficient way for storing sparse-matrices. It is particularly useful in situations where the matrix has many zero elements. Parallellising CSR matrix operations requires special treatment as not all elements in the matrix are readily available for independent processing. One solution is to process each row separately, making it straightforward to assign different rows to different threads. This approach requires the availability of atomic operations to extract good performance (eguly & Giles, 2012). For dynamic graphs stored in CSR, parallel algorithms can insert or delete edges concurrently. Lock-free or fine-grained locking mechanisms are employed to allow multiple threads to update the structure without significant contention(Eleni Alevra, 2020). However, this introduces complexity in managing numerous locks, increasing the risk of deadlock or priority inversion if not designed meticulously (Wheatman et al., 2024). Existing libraries often lack GPU support or are tied to proprietary platforms (e.g., CUDA (NVIDIA, 2022)). PIRO fills this gap by offering a novel solution (HTLF) by representing Sparse Matrices as Hash Tables (HT). It uses a hash function to compute an index (hash key) into an array of slots, where the corresponding value is stored. Insertion, deletion and lookup can be performed in amortized constant time (on average) independent of the number of non zero elements in the table, assuming a good hash function and well-sized table. It is well known that HT are generally more efficient than search trees or other lookup structures for these operations, especially when fast access is required.

Additionally the software offers:

- **Cross-platform CPU / GPU operation** via OpenCL.
- **Modular equation** solver.
- **Post processing** export function for viewing results (e.g Paraview).
- **Benchmarking** tools and diagnostics.

Algorithm - HTLF

When solving a linear system numerically, especially for large systems, directly computing the inverse is not recommended for numerical stability and efficiency. Iterative solvers are algorithms that find approximate solutions to linear systems, such as $Ax = b$, by starting with an initial guess and progressively improving it through repeated iterations. If approximate solutions are not desired, Direct methods, such as Gaussian elimination, LU decomposition, and Cramer's rule, compute the exact solution to a linear system in a finite number of steps, assuming no

rounding errors occur during computation. LU decomposition requires modification of the original matrix, which can be sparse by nature, into an Upper Triangular Matrix (UTM), which can also be sparse, before solving. This step adds, modifies and removes elements from the origianal matrix through repeated gaussian elimination steps (κ). If sparseness of the matrix is measured by the number of non zero elements (nnz), then

$$\mu = nnz/N^2$$

$$\bar{n} = \kappa/N^2$$

$$\lambda = nnz@\kappa : 0/nnz@\kappa : N-1$$

where, $N = n_x * n_y * n_z$ if n_x, n_y, n_z are the number of grid cells in X, Y and Z direction respectively. nnz for an UTM obtained after gaussian elimination of a Laplacian operator for various cell grid sizes (N) is shown in the figure below.

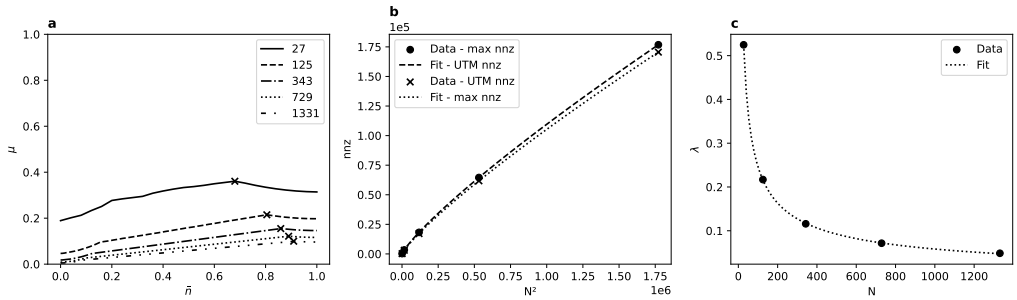

Figure 1: **(a)** Row filling for different N during gaussian elimination. 'x' marker denoting maximum nnz. **(b)** Curve fitting for nnz. **(c)** Curve fitting for the load factor.

The sparseness decreases initially, reaching a maximum, before finally increasing according to Figure 1.

Table size estimation

Choosing an appropriate table size for hashing is crucial for the performance of the algorithm. Since the trends are driven by (N), a curve fit for λ is performed as follows :

$$\lambda = a * N^b + c$$

The coefficients are $a = 3.3608$, $b = 0.5552$, $c = -0.014$.

Then,

$$TABLE_SIZE = nnz/(\lambda * \sigma)$$

where, σ is the scaling factor, providing more flexibility.

Hash function

A simple hash function is used :

$$key = index \ \% \ TABLE_SIZE$$

where,

$$index = row\ number * N + column\ number$$

DRAFT

Method

1. The sparse matrix is initially traversed to identify indices where extra elements are to be added. They are assigned a key of -1 in the HT. The HT is then copied to the device.
2. The gaussian elimination step is performed on every element in parallel by the kernel. Resulting zero elements are concurrently set to a key of -2. The HT is copied back to the host.
3. Steps 1. and 2. are repeated $N - 1$ times.

For a large enough $TABLE_SIZE$, steps 1. and 2. are O(1) at best. Sometimes traversal performed in step 1 might need extra probing (upon encountering -2 before -1) during insertion due to open addressing.

Performance

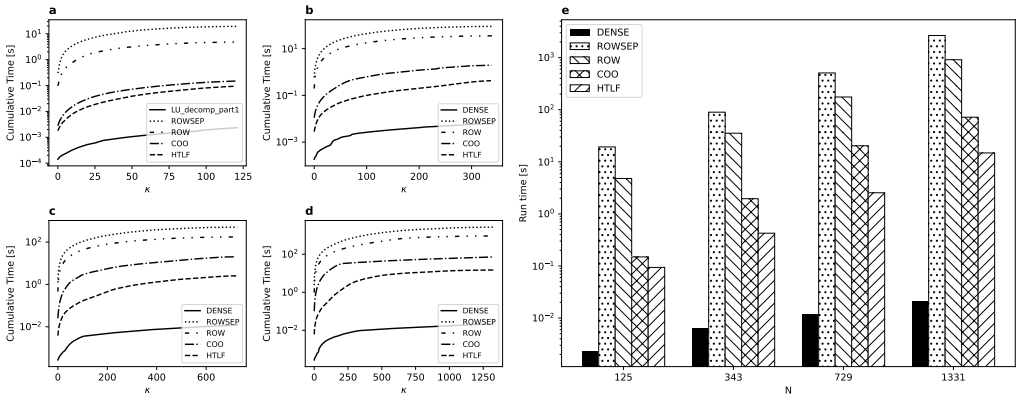

Figure 2: Cumulative run times across gaussian elimination steps for different N. **(a)** N = 125. **(b)** N = 343. **(c)** N = 729. **(d)** N = 1331. **(e)** Total run times. HTLF uses { } = 0.7.

Table 1: Run times and space complexities of various algorithms generating a UTM for a 7 x 7 x 7 grid Laplacian on an AMD Radeon Pro 5300M; $factor$ = percentage of N^2.

Study	**Time [s]**	**Host memory** [int, float]	**Device Memory** [int, float]
DENSE	0.0062	N + nnz + 1, N^2 + nnz	1, 3 * N^2
ROWSEP	89.6593	N + nnz + 1, 2 * N + nnz	N + factor + 1, 3 * N + factor + 1
ROW	35.3817	N + 2 * n + nnz + 1, N + 2 * n + nnz	3 * N + factor + 1, 3 * N + factor
COO	1.9186	N + 2 * nnz + 1, 2 * nnz	nnz + N + 1, nnz
HT	10.7566	TABLE SIZE + 3, TABLE SIZE	TABLE SIZE + 3, TABLE SIZE
HTLF (σ=0.7)	0.4587	TABLE SIZE + 3, TABLE SIZE	TABLE SIZE + 3, TABLE SIZE

DRAFT

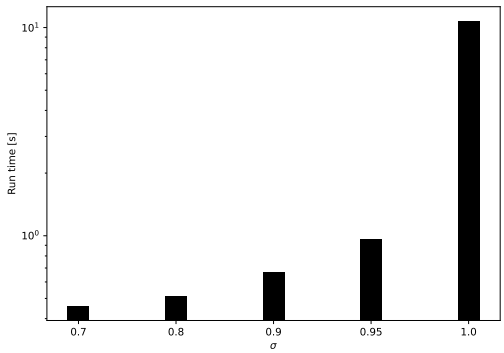

Figure 3: Run times for different σ.

Figure 3 shows performance improvements with reducing σ. The ideal value for σ can be chosen based on use case.

Conclusion

Hash table representaton of sparse matrices for executing operations using parallel processing gives flexibility to prioritize balance between time and space efficiency. This is benenifical as the method can be adapted easily based on use ase and hardware.

Scope for future work

A simple hash function was used to calculate key values. Alternative hash methods and its effect on various differential operators needs to be explored.

References

eguly, I. R., & Giles, M. (2012). Efficient sparse matrix-vector multiplication on cache-based GPUs. *IEEE Xplore*, 1–12. https://doi.org/10.1109/InPar.2012.6339602

Eleni Alevra, D. M., Christian Menges. (2020). A parallel packed CSR data structure for large-scale dynamic graphs. In *GitHub repository*. GitHub. https://github.com/domargan/parallel-packed-csr

NVIDIA. (2022). *cuSPARSE library*. NVIDIA Corporation. https://docs.nvidia.com/cuda/pdf/CUSPARSE_Library.pdf

Wheatman, B., Burns, R., & Xu, H. (2024). Batch-parallel compressed sparse row: A locality-optimized dynamic-graph representation. *IEEE Xplore*, 1–8. https://doi.org/10.1109/HPEC62836.2024.10938461

DRAFT